

KV 复用只有在节省的 Prefill 能穿过放置、传输与控制之后才真正成立。

本文把 KV cache 复用聚焦为一个传输与控制问题：关键并不是“是否命中缓存”，而是兼容状态能否被找到、治理、路由、物化，并在重算更划算之前进入推理引擎。

范围：agentic serving、分布式 KV store、传输底座、CXL、NIXL、PrfaaS

证据：论文报告、官方文档、定价信号、推导下界、定性图示

语言：简体中文版本

0. 摘要

命中率只是中间变量。Serving operator 最终关心的是满足 SLO 的 goodput、P50/P95 TTFT、TPOT 稳定性、GPU 空转、单位有效 token 成本，以及跨请求复用的安全性。

KV 复用把 prefill 计算转换成状态管理问题。一个可复用 prefix 或 block 只有在系统回答了四个问题之后才有价值：状态是否兼容，状态在哪里，多快能变成 engine-ready，以及条件变化时如何 fallback。

```
reuse_value ~= p_hit * prefill_compute_saved
              - storage_cost
              - p_fetch * transfer_cost
              - lookup/control_cost
              - repair_or_fallback_cost
```

- tail_latency_risk
- quality_or_governance_risk

因此，本文把 KV reuse 作为传输与控制问题来读：cache hit 不是终点，真正的系统问题是如何让命中的 KV 状态在正确时间、正确格式、正确位置抵达 decode 或后续 prefill。

下，一个
Kimi
风格
FP8 KV
对象的
量级锚
点。

输的
想下
界，
未计
拷贝
排队
碎片
化。

1. 数据范围与方法

这是一份传输聚焦的技术报告，不是完整学术综述。纳入对象的标准是：它是否改变 KV 状态跨服务、跨存储层、跨节点、跨 rack 或跨集群边界时的放置、移动、物化、路由或治理方式。

范围类别	纳入内容	只作背景
传输与物化	外部 KV store、prefill/decode handoff、RDMA/NIXL、NVMe-oF、GPU direct path、CXL/远端内存池。	单机 kernel 优化、纯本地 page allocator。
控制面	KV-aware routing、cache affinity、metadata index、admission、fallback、SLO-aware transfer planning。	与 KV 无关的一般负载均衡或批处理调度。
复用合同	exact prefix、modular chunk、semantic/approximate reuse、跨模型或 agent 状态交换中的传输可行性。	只返回旧答案的 response cache。
证据类型	原论文数字、官方文档、官方定价、硬件规格、推导下界和定性图示。	没有来源的供应商宣传或无法复核的二手数字。

定性图示用于帮助读者把握系统 trade-off，不应读作 benchmark。定价和产品文档是时间点信号，引用具体价格或产品限制前应重新核对官方页面。

2. Claim 与证据账本

主要论断需要可审计：每个 claim 都标明证据类型、适用条件以及应如何解读。

本文使用的 claim	证据类型	条件与限制	来源线索
KV 复用应以 SLO-qualified goodput、TTFT/TPOT 稳定性和 fleet utilization 判断，而不是只看命中率。	Sourced inference	来自 serving 系统论文和产品系统的共同趋势；不是单个公式证明。	Mooncake、DistServe、Splitwise、vLLM/Mooncake Store。
更大的可达 KV 池会提高 agentic trace 的命中机会，但更深层 tier 会引入移动税。	Reported + illustrative	趋势成立需要 workload 有重复结构；图中曲线是定性直觉。	vLLM/Mooncake Store、LMCache、CacheGen。
公开 cached-input 折扣说明复用有经济价值。	Official pricing	仅说明价格表奖励 cache-friendly 行为；不说明内部实现一定暴露 raw KV。	OpenAI API pricing、DeepSeek Models & Pricing。
Mooncake TE、Beluga、Dynamo/NIXL、PrfaaS 不应只用单一 speedup 数字比较。	Comparative synthesis	它们移动的边界不同，分别偏 multi-tier transfer、CXL memory semantics、direct handoff、cross-cluster prefill。	对应论文/官方文档。

3. 为什么 KV reuse 变成了系统问题

KV-cache 工作已经从本地内存布局扩展为面向传输的系统问题：可复用状态开始跨越服务、存储层、主机、rack，甚至模型和 workflow 边界。

图 1. KV cache 从单线优化走向多分支 serving 基础设施

这是报告语料中的指示性下界，不是 bibliometric claim。重点是 2025-2026 后分支变多。

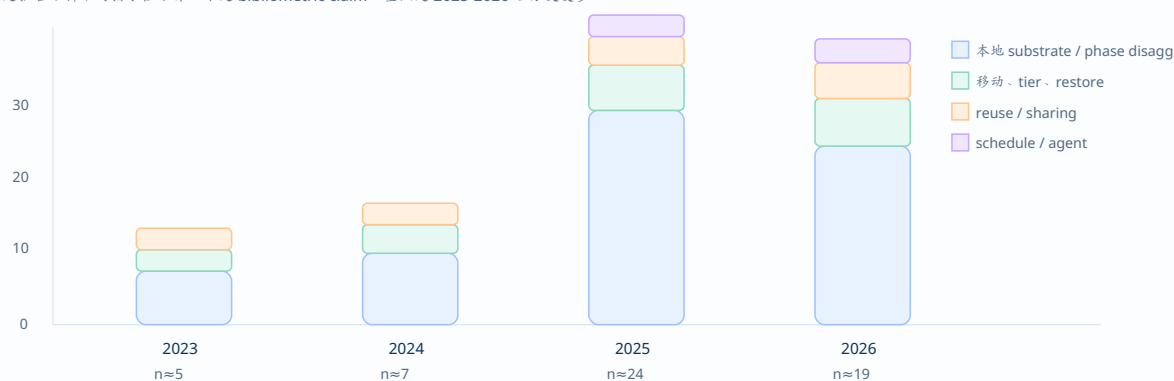


图 1. 指示性信号图。到 2025-2026 年，KV-cache 不再只是本地显存管理，而是延伸到存储、网络、路由、产品 API 和 agent 通信。

PagedAttention 让 KV layout 成为系统抽象；Splitwise 和 DistServe 把 prefill-to-decode handoff 变成执行边界；Mooncake 把生产平台组织成 KV-centric serving；LMCache、Mooncake TE、Dynamo/NIXL 和 llm-d 把外部 KV 移动与路由产品化；DroidSpeak、KVCOMM、Q-KVComm 等工作继续把 KV 推向跨模型和多 agent 通信。

4. 经济动机：cache-friendly 行为已经被定价

公开 cached-input discount 说明服务商给复用定价。但定价只能证明经济表面，不能证明内部实现一定是 raw KV 暴露、跨节点 KV 传输或某个具体 memory tier。

图 2. 输入 cache miss / cache hit 价格比例

官方定价时间点信号；不代表内部实现。

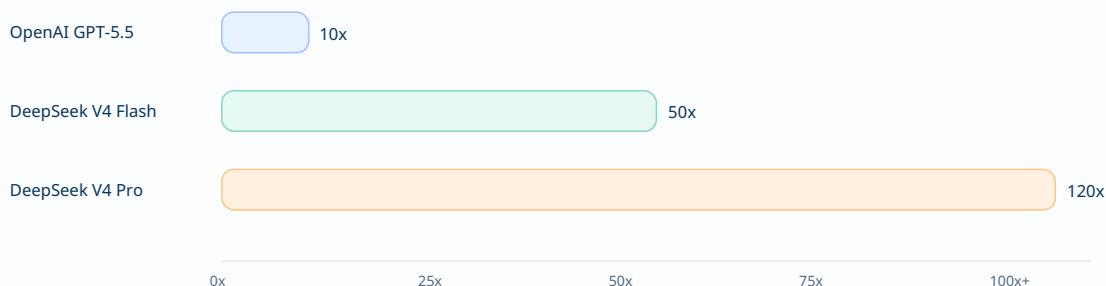


图 2. 价格比是经济信号：cached input 的低价鼓励稳定上下文、prefix 和重复工作负载，但不能单独推出具体 KV transport 架构。

模型/API	cache miss 输入价	cache hit 输入价	比例	备注
OpenAI GPT-5.5	\$5.00 / 1M input tokens	\$0.50 / 1M cached input tokens	10x	官方 API pricing; 时间点信号。
DeepSeek V4 Flash	\$0.14 / 1M input tokens	\$0.0028 / 1M cached input tokens	50x	官方 DeepSeek pricing; 价格可能变化。
DeepSeek V4 Pro	\$0.435 / 1M input tokens	\$0.003625 / 1M cached input tokens	120x	官方 DeepSeek pricing; 价格可能变化。

5. Operator 决策模型：route、move、repair、compress 或 recompute

KV reuse 的核心政策问题是 action policy。控制器不是最大化名义命中率，而是在 SLO、质量与治理约束下选择最便宜且安全的状态动作。

场景	可能更优动作	传输/控制需求	常见失败模式
兼容 KV 已在轻载 worker 附近。	Route request to state。	新鲜 location index、负载信号、tenant policy、cache-affinity scoring。	过度 affinity 形成 hotspot。
远端 KV 可用但对象大、碎片多。	Move state to request 或选择部分重算。	scatter-gather transfer plan、direct GPU path、completion semantics。	传输尾延迟超过重算。
approximate hit 可用但存在偏差。	Reuse + selective repair。	compatibility metadata、repair budget、quality fallback。	质量漂移或 repair 成本失控。
冷 tier 中有长上下文 KV。	Compress / prefetch / delayed materialization。	tier-aware latency model、admission、deadline-aware prefetch。	冷启动导致 TTFT 尾部爆炸。
无法确认安全复用。	Recompute。	明确 fallback path 和审计记录。	盲目复用带来 tenant 或模型兼容风险。

6. 结构视角：把可复用上下文编译成 SLO-aware 状态动作

跨层故事不是松散生态图，而是一条编译路径：应用与模型结构产生候选 KV 对象，控制器验证并选择动作，memory/transport 底座决定这些动作是否可行。

图 3. 从 reusable context 到具体状态动作



关键判断：状态是否值得保留、是否可以移动、是否必须修复、是否应当重算。

图 3. 传输优化只有在上游对象身份、兼容性和动作语义清晰时才有抓手。

7. 请求路径：每一次命中都会产生物化义务

Transport 不是调度之后的附录。它限制哪些 cache-aware 决策真的有利：一个迟到、碎片化、CPU-bound 或不安全的 hit，可能提高命中率却降低 goodput。

1. 暴露

应用区分 stable prompt、tool schema、workspace memory、dynamic tail、tenant policy 和 reuse hint。

2. 查找

Router 对 token block/chunk 做 hash；metadata plane 检查模型、tokenizer、layout、RoPE、dtype、tenant 和 TTL。

3. 决策

选择 route-to-state、fetch-state、prefill-missing、repair 或 recompute。

4. 传输

KV object 转化为 batch/scatter-gather plan，绑定 memory descriptor 和 completion semantics。

5. 物化

状态进入引擎可读的 page/block；decode 消费 live KV 并追加新 KV。

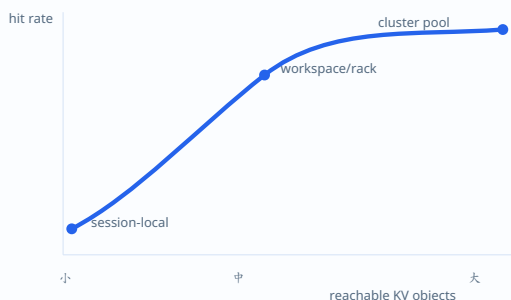
6. 发布

请求完成后发布 residency、hit/miss、latency、tier pressure 和治理事件。

8. 定量直觉：更大的池提高命中率，更深的层级增加移动税

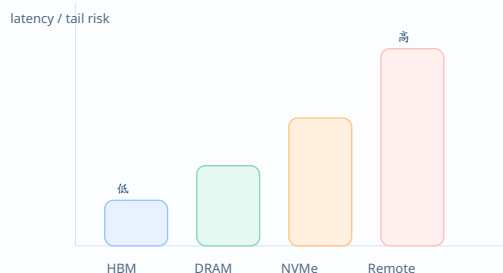
Transport-centered reading 需要同时看两条轴：可达 KV 容量可能提高 agentic trace 命中率；更多 tier 会带来 latency、控制复杂度和尾部风险。

图 4. 可达 KV 池与命中率直觉



定性图示：重复结构越稳定、可达对象越多，命中机会越高，但实际曲线由 workload 决定。

图 5. 层级越深，移动税越高



定性图示：对象越远、越冷、越碎片化，传输和控制成本越可能吃掉 prefill saved。

REPORTED

3.8 GB

100K-token 级 KV
footprint 量级。

DERIVED LOWER
BOUND

76 ms

3.8 GB / 400 Gb/s
的理想传输下界。

REPORTED

3.8x

vLLM ×
Mooncake Store
在 agentic trace
中的 throughput
anchor。

ILLUSTRATIVE

tier tax

多 tier 带来
setup、copy、
queue 和 control
overhead。

9. KV 对象 ABI：先让状态可见，再谈聪明传输

传输层无法优化匿名 tensor bytes。它需要最小 KV object contract：身份、兼容性、位置、有效性、策略和完成语义。

字段组	代表字段	为什么重要
Identity	object id、block hash、request/session/workflow id、producer engine id。	命名可复用状态，避免重复发现。
Model compatibility	model id、tokenizer、layer/head layout、RoPE offset、dtype、quantization、compression。	防止错模型、错位置、错表示复用。
Location	tier、host/GPU、memory descriptor、registration handle、object size、fragment list。	让 scheduler 估计移动成本并生成 transfer plan。
Validity	TTL、provenance、tenant、privacy label、reuse confidence、staleness。	支撑治理、安全和 fallback。
Completion	async transfer id、partial completion、repair required、engine-ready signal。	让命中的 KV 变成可消费状态，而不是只存在于 metadata。

10. Mooncake TE 作为 transfer-plan 底座

Mooncake TE 最好不要只读作 RDMA。RDMA 是 backend；更高层贡献是 registered segments、异步 batch transfer、拓扑感知路径选择，以及可被 Mooncake Store / vLLM connector 组合成 KV 物化的 substrate。

Case-study 维度	解读
Problem	跨节点、跨 tier 的 KV 物化需要避免 CPU copy、碎片化 transfer 和 topology-unaware path。
Boundary moved	从 engine-local KV 到 distributed multi-tier KV service。
Data path	RAM、VRAM、NVMe endpoint 上的 segment；BatchTransfer；RDMA/TCP 等 backend。
Control dependency	需要上层 store/router 决定哪些 KV object 值得移动、何时移动、移动到哪里。
Best-fit regime	agentic trace、长上下文、高 prefix reuse、可用 RDMA/GPU direct 的集群。
What not to infer	TE 自身不是完整 reuse policy，也不自动解决兼容性和治理。

11. Beluga: CXL 是 substrate, Beluga-KVCache 才是 KV-aware 层

Beluga 最清楚的读法是两步：先说明 RDMA 式移动对碎片化 KV 不友好，再在通用 CXL memory semantics 之上增加 KV-aware indexing、copy 和 control。CXL pooling 本身不是 KV-block aware。

图 6. CXL memory semantics 与 KV-aware 层的区别



图 6. CXL 解决的是内存语义和池化可达性；KV 感知来自上层 layout、index、copy 和 policy。

12. Dynamo / NIXL: 偏直接 handoff, 而不是完整 KV 生命周期

Dynamo 的传输强项是生产化 disaggregated serving 中基于 NIXL 的直接 VRAM-to-VRAM KV transfer。它最适合在快速 NVIDIA-oriented fabric 内完成 prefill 到 decode 的 KV handoff。

图 7. Dynamo / NIXL 的直接 prefill-to-decode handoff

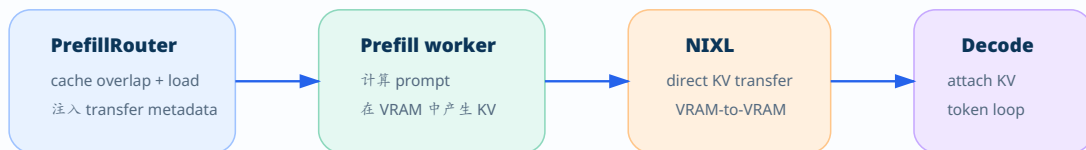


图 7. Dynamo/NIXL 的核心价值是快路径 handoff; 长寿命、多 tier、跨 tenant 的 KV 生命周期需要额外 store/control plane。

13. PrfaaS: 跨集群 prefill 是有条件的边界外移

PrfaaS 的意义是追问 PD boundary 可以外移多远。答案不是总是向外: remote prefill 只有在模型侧 KV 缩小、选择性 admission、带宽感知路由和尾部风险控制同时成立时才赢。

图 8. PrfaaS: 当 returned state 足够小时, prefill 可以跨集群服务化

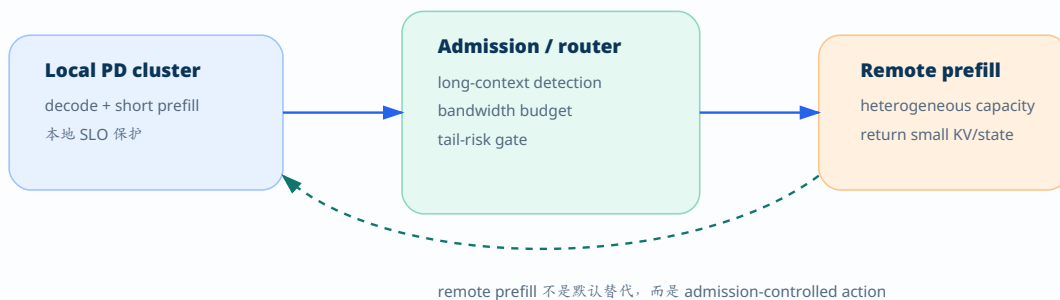


图 8. 跨集群 prefill 把 phase boundary 推出单集群, 但必须由 bandwidth 和 tail-risk 约束。

何时成立

- 远端 prefill 端有明显闲置或更便宜能力。
- 返回 KV/state 足够小，或可以选择性返回。
- WAN / cross-cluster 带宽与抖动不会破坏 P90/P95 TTFT。
- 本地 decode 仍能稳定消费返回状态。

何时不成立

- 上下文短，重算便宜。
- 返回状态大，链路抖动高。
- prefix cache 分布不均，远端命中不稳定。
- 跨租户治理和审计成本超过收益。

14. 对比图谱：按移动的边界比较，而不是按单个 speedup 数字比较

Mooncake TE、Beluga、Dynamo/NIXL 和 PrfaaS 回答的是不同传输问题。更有用的比较维度是 boundary、data-path proposition、control dependency、best-fit workload、failure mode 和 evidence type。

工作	移动的境界	数据路径主张	控制面依赖	最佳适用场景	压力点 / 非目标
Mooncake TE / Store	engine-local KV -> distributed multi-tier KV service	segments、 batch transfer、拓扑 感知、多 backend。	KV store、 router、 admission、tier policy。	长上下文、 agentic trace、高 复用集群。	TE 不是完整 reuse policy。
Beluga	RDMA-pool mental model -> CXL- attached KV- aware memory layer	CXL load/store 语义上叠加 KV- aware copy/ index。	KV block layout、 scheduler hooks、copy engine。	碎片 KV、远端内 存池、CPU/GPU polling 成本高的 场景。	CXL pooling 本身不懂 KV。
Dynamo / NIXL	prefill worker -> decode worker fast handoff	VRAM-to-VRAM direct transfer。	PrefillRouter、 overlap/load scoring、transfer metadata。	快速 fabric 内 phase- disaggregated serving。	不是完整 长寿命多 tier KV store。
PrfaaS	intra-cluster PD -> cross- cluster prefill service	远端 prefill + selective returned state。	admission、 bandwidth-aware routing、tail-risk guard。	长上下文、异构资 源、prefill capacity burst。	网络抖动 可能主 导。

15. 评估纪律与治理考虑

每个 transport claim 都应回答 operator 问题：在移动、控制、质量和治理成本之后，这个机制是否提高了满足 SLO 的 goodput？

评估维度

- Boundary moved: 本地 page、外部 store、PD handoff、CXL pool、cross-cluster service。
- Workload regime: agentic turn trace、RAG/document reuse、long-output decode、multi-tenant prefix。
- Hit denominator: request、block、token、byte、SLO-qualified hit。
- Transfer metric: p50/p95/p99、setup overhead、copy count、CPU/GPU stall。

治理风险

- 跨租户泄漏。
- 模型/tokenizer/RoPE/layout 不兼容。
- 上下文变更后的 stale state。
- approximate reuse 的质量漂移。
- 缺少审计、删除和撤销语义。

报告口径

- Reported: 来自论文或官方报告。
- Derived: 公式推导或带假设下界。
- Illustrative: 帮助理解的定性图示。
- Sourced inference: 由多个来源共同支持的系统判断。

Source trail 与延伸阅读

来源按角色分组。产品页面、定价页面和硬件规格是时间点信号，引用具体价格或产品限制前应重新核对。本文定价来源在 2026-06-05 版本中核对。

系统论文与 preprint

- **S3** - Mooncake FAST'25 paper。

项目、文档和价格信号

- **S1** - vLLM × Mooncake Store blog。

- **S8** - Beluga: CXL-Based Memory Architecture for LLM KVCache。
- **S9** - Prefill-as-a-Service paper。
- **S11** - Agentix NSDI'26 paper。
- **S12** - DroidSpeak NSDI'26 paper。
- **S13** - DistServe / Splitwise / CacheGen / LMCache 相关论文。
- **S2** - Mooncake Transfer Engine / Store documentation。
- **S4** - NVIDIA Dynamo / NIXL documentation。
- **S5** - OpenAI API pricing。
- **S6** - DeepSeek Models & Pricing: https://api-docs.deepseek.com/quick_start/pricing。
- **S7** - GPU / NIC / CXL 相关硬件规格。